FEATKT: FEATURE-DRIVEN MULTI-STEP KNOWLEDGE TRACING

Sahishnu Durgam[1] **Prajith Pandrate**[2]

[1]Academy of Engineering and Technology [2]University of Illinois Urbana–Champaign

Introduction

- **Knowledge tracing (KT) predicts whether a student will answer a question correctly based on their previous learning history.**
- Most KT models are evaluated using one-step-ahead prediction, where the correct answer to each previous question is revealed before making the next prediction. However, many real learning scenarios require predicting an entire future sequence without receiving per-question feedback.
- pyKT, a benchmark platform for KT models, calls this **non-accumulative multi-step (NAMS)** prediction. Once prediction begins, a sequence model receives no new responses to update on, so its main advantage over a simpler model disappears.

Methodology

- This study uses the EdNet-KT1 data as distributed through the Riiid AIEd Challenge. 5,000 students were randomly selected using a fixed seed of 42.
- The data were preprocessed using the same pipeline as pyKT. Each student interaction was expanded so that every row represented one question–concept pair. Student sequences were padded or shortened to a maximum length of 200 interactions.
- The selected users were divided into 4,000 training users and 1,000 testing users at the user level, meaning that no student's interactions appeared in both sets.
- For a student sequence of length L and a prefix ratio r, the model is given the first $\lceil rL \rceil$ interactions and their correct or incorrect labels. It must then predict every remaining interaction without receiving any additional feedback.

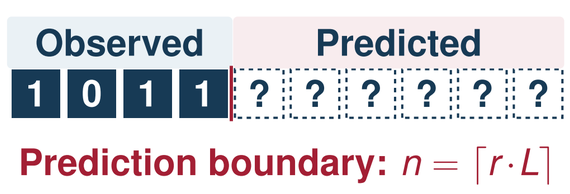

Fig. 1. The NAMS protocol at prefix ratio $r = 0.4$ for a sequence of length $L = 10$. Correctness labels are never revealed after $t = n$.

- Performance was measured using AUC, averaged across five prefix ratios.

Proposed Model: FeatKT

FeatKT is a gradient-boosted decision tree model built using LightGBM. FeatKT predicts each interaction using five carefully selected features:

1. **Concept identifier** – represents the concept or skill tested by the question.
2. **Question difficulty** – measures how difficult a question is based on the percentage of training students who answered it incorrectly.
3. **Log elapsed time** – the logarithm of one plus the time spent answering the previous question, $\log(1 + t_e)$.
4. **Log lag time** – the logarithm of one plus the time between consecutive interactions, $\log(1 + t_l)$.
5. **Frozen prefix accuracy** – $a_u = \frac{1}{n} \sum_{i=1}^{n} y_{u,i}$, computed once at the prediction boundary.

The value is calculated once at the end of the observed prefix and remains unchanged throughout the prediction period. Updating it during prediction would require information from future responses, which would violate the NAMS protocol.

Selected References

[1] A. T. Corbett and J. R. Anderson, "Knowledge Tracing: Modeling the Acquisition of Procedural Knowledge," *User Modeling and User-Adapted Interaction*, 1994.

[2] C. Piech et al., "Deep Knowledge Tracing," *NeurIPS*, 2015.

[3] Z. Liu et al., "pyKT: A Python Library to Benchmark Deep Learning Based Knowledge Tracing Models," in *NeurIPS*, 2022.

[4] Y. Choi et al., "EdNet: A Large-Scale Hierarchical Dataset in Education," *AIED*, 2020.

[5] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," *NeurIPS*, 2017.

Model Architecture

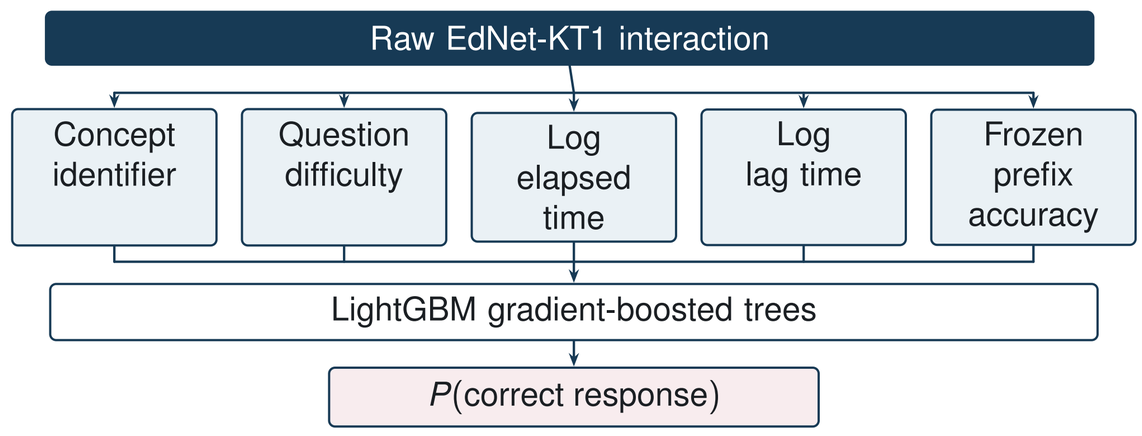

Fig. 2. FeatKT architecture. FeatKT does not use a sequence encoder, attention mechanism, or learned embeddings.

Results

Model	Model class	Avg. AUC
DKT	Recurrent	0.594
sparseKT	Sparse attention	0.646
simpleKT	Transformer	0.666
AKT	Difficulty-aware	0.666
Transformer + features	Transformer	0.726
FeatKT	**Boosted trees**	**0.747**

Table I. Average AUC across five NAMS prefix ratios.

- FeatKT achieved the highest average AUC of all the models tested and had the highest AUC at every single ratio.
- Providing the engineered feature set to a transformer increased its average AUC from 0.666 to 0.726.
- The features themselves have a large impact on performance.

Among the published KT models, AKT and simpleKT achieved the best performance, likely because they are the only baselines that explicitly include question difficulty. Baseline results should be read as default-configuration performance rather than as tuned upper bounds.

Validation

The label-shuffled model achieved an AUC of 0.5093, indicating that the preprocessing pipeline did not introduce target leakage.

Check	Result	Interpretation
Five-seed retraining	std. = 0.0001	Effectively deterministic
Label-shuffle control	0.5093 AUC	Chance level; no leakage
Train/test overlap	0 of 1,000	Splits are disjoint
Maximum feature gain	26.8%	No feature dominance

Table II. Validation checks for leakage and feature dominance.

A paired bootstrap over test users confirmed that FeatKT's advantage holds at every prefix ratio ($p < 0.001$).

Ratio	Gap	95% CI	FeatKT wins
0.2	0.024	[0.014, 0.034]	100.0%
0.4	0.027	[0.018, 0.037]	100.0%
0.6	0.019	[0.014, 0.025]	100.0%
0.8	0.022	[0.014, 0.029]	100.0%
0.9	0.012	[0.006, 0.020]	99.9%

Table III. Paired-bootstrap comparison with the feature-matched transformer.

Conclusion

- This study evaluated KT using pyKT's NAMS protocol, where models must predict a student's future responses without receiving per-interaction feedback. Under this setting, the results showed that the choice of input features had a greater impact on performance than the choice of model architecture.
- Only 5,000 EdNet users were included to reduce training time, and all models were evaluated on a single train-test split. Future work should evaluate FeatKT on the full EdNet dataset and other KT datasets, such as ASSISTments.